

Agente de Atendimento IA

Arquitetura, componentes e decisões de design de um agente de atendimento por IA agnóstico de canal — Telegram, WhatsApp (Meta Cloud API) ou qualquer webhook — com RAG, handoff para humano e auditoria de conversas.

Node.js + TypeScript

Claude / OpenAI

Voyage AI Embeddings

Telegram · WhatsApp · Web

Repositório: github.com/andre-rizzato/ai-customer-service-agent

Versão do documento: 1.1 — Outubro/2026 (revisão parcial, ver aviso abaixo)

Sumário

1. Visão geral e origem
2. Arquitetura
3. Fluxo de uma mensagem
4. Componentes e responsabilidades
5. Estrutura de pastas
6. Configuração (negócio vs. ambiente)
7. Base de conhecimento e RAG
8. Handoff para humano
9. Canais suportados
10. Testes
11. Como estender o projeto

Documento de 28/09/2026 — revisado só em parte em 09/10/2026

Este guia descreve a base original do projeto. Os trechos de RAG e de dados persistidos foram corrigidos, mas o que veio depois não está aqui: Key Vault, capacidades (AgentService), relay de handoff pelo Telegram, idioma do cliente, deploy automático pelo GitHub Actions, medição de custo e memória da conversa. Para o estado atual, veja [README.md](#), [docs/STATUS.md](#), [docs/RAG_QDRANT_MIGRATION.md](#), [docs/HANDOFF_RELAY.md](#), [docs/CUSTO_API.md](#) e o diagrama [docs/artifacts/fluxo-da-requisicao.html](#).

1. Visão geral e origem

Este projeto implementa, em código, o runbook "*Agente de atendimento por IA no WhatsApp, do zero à produção*" — mas de forma **agnóstica de canal**. A ideia central do runbook original é replicável para qualquer plataforma de mensagens: uma base de conhecimento fechada (o agente nunca inventa specs, preços ou prazos), transparência total quando perguntado se é uma IA, e um handoff limpo para um atendente humano quando a conversa sai do escopo.

Em vez de acoplar essa lógica ao WhatsApp, o projeto isola tudo que é específico de plataforma atrás de uma interface ([ChannelAdapter](#)), e tudo que é regra de negócio (RAG, prompt, handoff, log) em um **Orchestrator** único e compartilhado. O resultado: o mesmo comportamento de atendimento funciona hoje no Telegram e no WhatsApp, e amanhã em qualquer outro canal que implemente a mesma interface.

Por que isso importa

Se o objetivo é oferecer isso como serviço para múltiplos clientes/negócios, cada um pode rodar sua própria instância com seu próprio catálogo, tom de voz e canal preferido — sem duplicar nem uma linha da lógica de atendimento.

2. Arquitetura

Cinco peças, cada uma isolada atrás de uma interface própria — trocar a implementação de qualquer uma não exige mudanças nas outras:

1 · Canal (ChannelAdapter)

Recebe o webhook da plataforma, normaliza para `InboundMessage`, envia a resposta de volta. Implementações: Telegram, WhatsApp Cloud API, Web (REST simples para testes).

2 · Orquestrador

Pipeline central: rate limit → gatilho de handoff → RAG → prompt → LLM → log. Não sabe de onde a mensagem veio nem para onde a resposta vai.

3 · LLM (LLMProvider)

Gera a resposta em linguagem natural a partir do contexto recuperado — nunca da memória própria do modelo.
Implementações: Anthropic (Claude), OpenAI.

4 · Base de conhecimento (RAG)

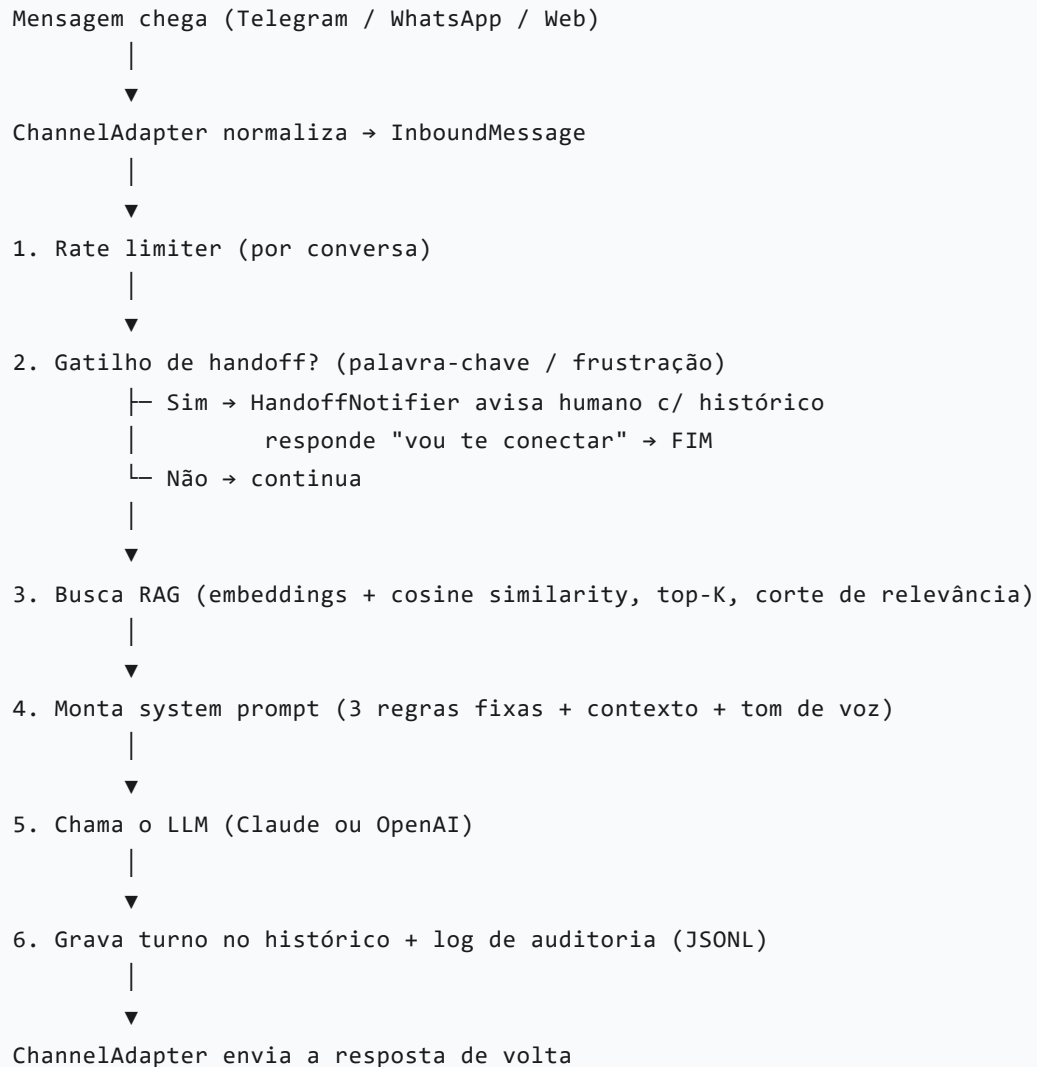
Catálogo, specs, políticas, preços — indexados como embeddings em um vector store local.
Implementações de embedding: Voyage AI (padrão), OpenAI, local (offline).

5 · Handoff (HandoffNotifier)

Regra de transferência + histórico anexado.
Implementações: console (dev), webhook (Slack/Discord/custom).

3. Fluxo de uma mensagem

Implementado em `src/orchestrator/orchestrator.ts`, método `handleMessage()` — o mesmo caminho para qualquer canal:



4. Componentes e responsabilidades

Peça	Arquivo	Responsabilidade única
Orchestrator	<code>src/orchestrator/orchestrator.ts</code>	Coordena o pipeline completo; único ponto de entrada (<code>handleMessage</code>) usado por todo canal.
Detector de handoff	<code>src/orchestrator/handoff.ts</code>	Classifica a mensagem como pedido explícito, frustração, ou nenhum gatilho — por palavra-chave configurável.
Prompt builder	<code>src/orchestrator/promptBuilder.ts</code>	Monta o system prompt com as 3 regras fixas + contexto recuperado.
Rate limiter	<code>src/orchestrator/rateLimiter.ts</code>	Janela fixa de mensagens por conversa, evita custo descontrolado em loop.
ConversationStore	<code>src/conversation/store.ts</code>	Histórico por conversa (JSON) + log de auditoria append-only (JSONL).
KnowledgeBase	<code>src/knowledge/knowledgeBase.ts</code>	Busca semântica com corte de relevância mínima ("regra de vazio").
FileVectorStore	<code>src/knowledge/vectorStore.ts</code>	Índice vetorial em arquivo JSON, similaridade de cosseno por força bruta.
Ingest	<code>src/knowledge/ingest.ts</code>	Script CLI que lê o catálogo e popula o vector store (<code>npm run ingest</code>).
LLMProvider	<code>src/llm/*</code>	Interface + implementações Anthropic/OpenAI, escolhidas por <code>LLM_PROVIDER</code> .
EmbeddingProvider	<code>src/embeddings/*</code>	Interface + implementações Voyage/OpenAI/local, escolhidas por <code>EMBEDDING_PROVIDER</code> .
HandoffNotifier	<code>src/handoffNotifier/*</code>	Interface + implementações console/webhook, escolhidas por <code>handoffNotifier</code> no config.
ChannelAdapter	<code>src/channels/*</code>	Interface + implementações Telegram/WhatsApp/Web.

Peça	Arquivo	Responsabilidade única
Server	<code>src/server.ts</code>	Sobe o Express, monta um Orchestrator e uma rota <code>/webhook/<canal></code> por adapter habilitado.

5. Estrutura de pastas

```
.
├── config/
│   └── agent.config.example.json # nome do negócio, tom, gatilhos de handoff
├── knowledge/
│   └── catalog.example.json # catálogo (produtos/serviços/políticas)
├── src/
│   ├── channels/ # Telegram, WhatsApp, Web
│   ├── conversation/ # histórico + log de auditoria
│   ├── embeddings/ # Voyage, OpenAI, local
│   ├── handoffNotifier/ # console, webhook
│   ├── knowledge/ # vector store, ingest, busca
│   ├── llm/ # Anthropic, OpenAI
│   ├── orchestrator/ # pipeline central
│   ├── config.ts # carga e validação (.env + agent.config.json)
│   ├── server.ts # endpoint HTTP
│   └── types.ts # tipos compartilhados
├── scripts/simulate.ts # chat de teste via terminal
├── tests/ # suíte automatizada (Vitest)
├── .vscode/launch.json # configs de debug
└── .env.example # template de variáveis de ambiente
```

6. Configuração: negócio vs. ambiente

O projeto separa deliberadamente dois tipos de configuração, cada um com seu próprio arquivo e propósito:

	<code>config/agent.config.json</code>	<code>.env</code>
Natureza	Configuração do negócio	Segredos e ambiente de execução
Exemplos	Nome do negócio, tom de voz, palavras de handoff, top-K do RAG	Chaves de API, tokens de canal, portas, caminhos de dados
Versionamento	Pode ser versionado por cliente/negócio	Nunca commitado (.gitignore)
Quem define	O negócio, na Fase 0 (briefing)	Quem opera a infraestrutura

Os dois são carregados e validados (com `zod`) uma única vez, em `src/config.ts`, na inicialização do processo — qualquer campo ausente ou mal formatado falha imediatamente, com mensagem clara, em vez de quebrar silenciosamente no meio de uma conversa.

7. Base de conhecimento e RAG

Cada linha do catálogo (`knowledge/catalog.example.json`) representa um produto, serviço ou política, no formato:

```
{ "id": "filtro-fx200", "title": "Filtro FX200", "content": "specs, preço, garantia, FAQ..." }
```

O script `npm run ingest` lê esse catálogo, gera um embedding por item (via Voyage/OpenAI/local) e grava tudo numa coleção do **Qdrant** (local em dev, Azure Container Apps em produção). Em tempo de resposta, a busca combina similaridade semântica (Qdrant) com palavra-chave (BM25), funde os dois rankings (RRF) e reordena os candidatos com o rerank da Voyage. Os `topK` trechos acima de `minRelevanceScore` entram no prompt.

A busca roda primeiro com a pergunta do cliente. Só quando o melhor trecho fica abaixo de `hydeSkipScore` ela tenta de novo com **HyDE**: uma passagem hipotética, gerada pelo LLM no estilo do catálogo, ajuda em perguntas indiretas. Detalhes em `docs/RAG_QDRANT_MIGRATION.md` ; custo e números medidos em `docs/CUSTO_API.md` .

Memória da conversa: o LLM recebe só os últimos `historyWindowTurns` turnos (padrão 15). O que sai da janela volta como um resumo acumulado e como trechos antigos relevantes (`src/conversation/memory.ts`). Assim o custo de uma conversa longa não cresce sem limite, e o bot continua lembrando do que foi dito no começo.

Regra de vazio

Se nenhum item passar no corte de relevância, o prompt informa explicitamente "nenhum trecho relevante encontrado" — é isso que permite à regra fixa nº 1 instruir o modelo a admitir que não sabe, em vez de inventar uma resposta.

8. Handoff para humano

Roda **antes** da busca RAG e da chamada ao LLM (mais barato e mais previsível que deixar o próprio modelo decidir). Dois gatilhos, configuráveis em `agent.config.json` :

- **Pedido explícito** — lista `handoffKeywords` (ex.: "falar com atendente").
- **Frustração** — lista `frustrationKeywords` (ex.: "péssimo", "não aguento").

Quando dispara, o `HandoffNotifier` configurado recebe o histórico completo da conversa (não só a última mensagem) e o usuário recebe uma resposta padrão avisando que será conectado a um atendente. A regra 3 do prompt (ver Seção 9 do guia de deploy) reforça o mesmo comportamento como segunda camada, para casos que a lista de palavras-chave não cobrir.

9. Canais suportados

Canal	Arquivo	Observação
Telegram	<code>src/channels/telegram.ts</code>	Webhook simples, secret token opcional para autenticar a Meta/Telegram.
WhatsApp	<code>src/channels/whatsapp.ts</code>	Meta Cloud API oficial — GET para verificação, POST para mensagens.
Web	<code>src/channels/web.ts</code>	REST simples (<code>POST /webhook/web</code>) para testes locais ou widget próprio.

Adicionar um novo canal (Instagram, SMS, etc.) significa implementar a interface `ChannelAdapter` e registrá-lo em `src/server.ts` — nenhuma linha do Orchestrator muda.

10. Testes

A suíte automatizada (`npm test` , Vitest) cobre a parte determinística da matriz de testes da Fase 5 do runbook original — detecção de handoff, montagem do prompt, rate limiting, ranking da busca (BM25/RRF), relay de handoff, memória da conversa e medição de custo — sem depender de nenhuma API externa. A qualidade com LLM de verdade é medida sob demanda pelo eval RAGAS e pelo teste de conversa longa (`eval/README.md`). O script `npm run simulate` complementa isso com um chat interativo no terminal, para rodar manualmente os casos que dependem do LLM responder (pergunta dentro/fora do escopo, "você é IA?", reclamação).

11. Como estender o projeto

Quero...	Faço isso
Trocar de LLM	Mudar <code>LLM_PROVIDER</code> no <code>.env</code> , ou implementar <code>LLMProvider</code> e registrar em <code>src/llm/index.ts</code>
Trocar de embeddings	Mudar <code>EMBEDDING_PROVIDER</code> , ou implementar <code>EmbeddingProvider</code> em <code>src/embeddings/</code>
Adicionar um canal novo	Implementar <code>ChannelAdapter</code> em <code>src/channels/</code> e registrar em <code>src/server.ts</code>
Crescer o catálogo	Já suportado: a busca roda no Qdrant (<code>src/knowledge/qdrantStore.ts</code>). Reindexe com <code>npm run ingest</code>
Saber quanto cada cliente custa	<code>npm run usage:report</code> (lê <code>data/usage-log.jsonl</code> ; ver <code>docs/CUSTO_API.md</code>)
Notificar handoff em outro lugar	Implementar <code>HandoffNotifier</code> (ex.: e-mail, Teams) em <code>src/handoffNotifier/</code>